Start here

What this project is, for somebody who has never seen it.

Modern software can be built out of **AI agents**: small programs, each given a job description in plain English, that talk to each other to answer a question. One agent looks things up. Another does the arithmetic. A third decides who to ask. Together they are called an **agent network**.

Cognizant AI Lab publishes a framework called **neuro-san** for building these. It has a clever feature: describe the job you want in one sentence and it will design a whole network for you, automatically, in about five seconds.

> **The problem this project exists to solve**
>
> It designs **one** network, and it never checks whether that network is any good.
>
> There is no scoring function anywhere in the framework. So nobody can answer simple questions like: is a nine-agent design better than a five-agent one for this job? Should each agent use the expensive AI model or the cheap one? Are the instructions it wrote actually clear? It is design with no measurement — like an architect who draws one building and never checks whether it stands up.

This project adds the missing half. It does two things:

It measures	Runs a network against 17 real questions and records three things: how many it got right, how much it cost, and how many agents it needed.
It searches	Automatically tries thousands of variations of that network to find a better one — and keeps doing it, every hour, without anybody asking.

The search method is called **ESP**, and it is Cognizant AI Lab's own invention. This project applies their method to their own framework, which nobody had done.

The idea in one picture

Why the clever part is clever.

Testing a real car shape in a wind tunnel is slow and expensive. So engineers do this instead:

```
1.  Test a handful of shapes in the real wind tunnel.     SLOW, COSTLY

2.  Use those results to build a cheap computer simulator
    that guesses how a shape will do.

3.  Try TEN THOUSAND shapes in the simulator overnight.   FREE, INSTANT

4.  Put only the best few back in the real wind tunnel.   SLOW, COSTLY
                                                          ...but only
                                                          a few of them

5.  Feed those new real results back into the simulator.  Repeat.
```

That is exactly what this project does, with agent networks instead of car shapes. The wind tunnel is a real test run, which takes about **eight minutes** and costs real money. The simulator is a small prediction model. And the numbers are not an analogy — they were measured on this project:

	Real test (the wind tunnel)	Prediction (the simulator)
Time for one network	about 8 minutes	**0.000064 seconds**
Time for 2,000 networks	about 267 hours	**0.13 seconds**
Cost	real money, every time	**nothing at all**

> **This is the whole point**
>
> Trying two thousand designs would take eleven days and a large bill. Trying two thousand *predictions* takes a tenth of a second and costs nothing. You only pay for the few the prediction says are worth it. That trade is what makes the search affordable — and it is why this is ESP rather than ordinary trial and error.

How it actually works

Six steps, in order, in plain English.

1. Build a pretend company nobody has heard of

A fictional logistics firm: 24 depots, 40 contracts, 60 incident reports, 124 documents. It is invented on purpose. If we used a real company, the AI might already know the answers from its training and would appear to do well without looking anything up. Nothing about this company exists, so the only way to answer is to go and read.

2. Write 17 questions with known answers

They range from easy ("who manages depot D03?") to hard, needing four documents joined together ("incident INC-4407 hit a contract, which is served by a depot — who manages that depot?"). Because the company and the questions come from the same generator, every answer is correct by construction. Nobody hand-wrote an answer that might be wrong.

3. Score a network by running it

Give the network all 17 questions and count: how many correct, how many words of AI it burned through, how many agents it used. Correctness matters most — a cheap network that answers nothing is worthless — but cost and size count too.

4. Make small changes and see what happens

Seven kinds of change: add an agent, remove one, rewire who talks to whom, split one agent into two, merge two into one, change who is allowed to search the documents, and switch an agent to a cheaper or more expensive AI model. Each change is checked for sanity first, and a broken one is **thrown away rather than patched up** — patching it would mean testing a different change from the one we made.

5. Learn to guess the score without running anything

Look at the shape of a network — how many agents, how deep, who talks to whom, which models — and predict its score. This is the simulator from the wind tunnel picture. It is trained on the real results collected so far, so it starts out poor and gets better as more results arrive.

6. Try thousands, pay for a few, repeat forever

Generate thousands of changed networks, predict all their scores for free, keep the best few, and pay to really run only those. Their real results go back into training the predictor, which then makes better guesses next time.

What it found

The result that makes the whole thing worth doing.

Three starting network designs were run for real against all 17 questions:

The design	Questions right	Cost (words of AI)	Agents
The shape neuro-san's own designer produces	**82%**	**278,532**	4
A flat pair of agents	**82%**	326,364	3
One agent doing everything	**82%**	**396,378**	1

Read that table again

All three got **exactly the same number of questions right**. But the most expensive one cost **42% more** than the cheapest — 118,000 extra words of AI for zero extra correct answers.

The cost difference is the real finding. On a real system running thousands of requests a day, that is a 42% bill for nothing — and there is currently no way to see it, because the framework has no way to measure any of this.

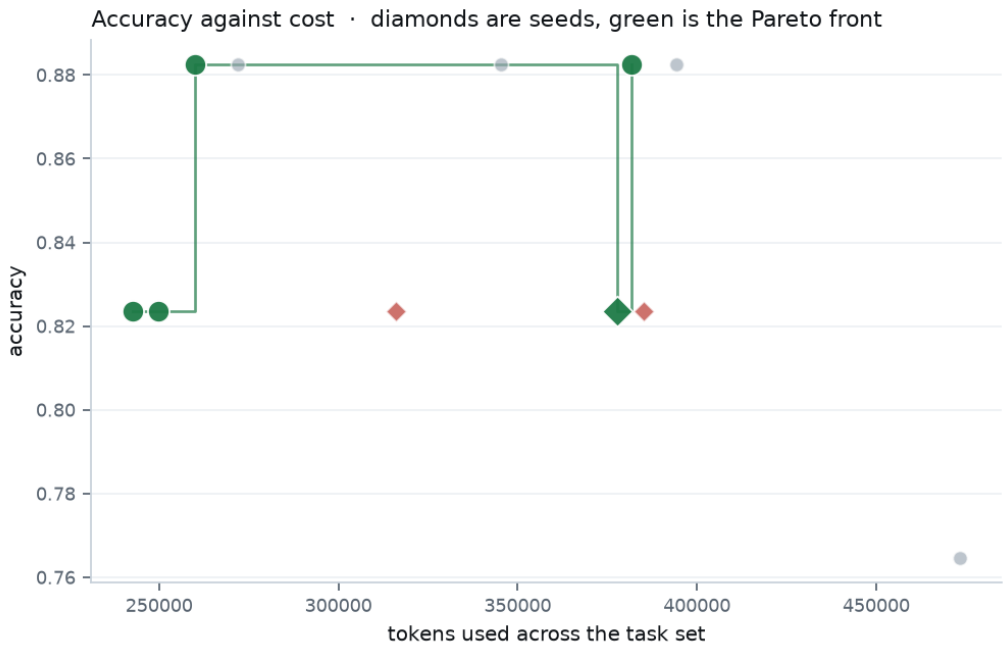
And now read the small print

The matching scores are a coincidence, and this document used to present them as a result. Each design failed three of the seventeen questions — but for three different reasons. The one-agent design **crashed** on all three and never answered them at all. The four-agent design **ran out of time** on two and got one wrong.

Counting a crash as a wrong answer is what made them look equal. Counting only the questions each one actually finished, the one-agent design got **everything** right and the four-agent design did not. What the shape of the network changed was how often it *finished*.

That is a more interesting finding than the one it replaced, and a weaker one: it rests on three measurements.

That gap is the entire opportunity. It is also, embarrassingly, only visible because of a bug that was found and fixed partway through: the cost measurement was scaled wrongly, so every design looked equally expensive. Until that was fixed the project would have reported "no difference" and been wrong.



Every network measured. The further right, the more expensive. The further up, the more correct. You want the top left.

Why it runs by itself, forever

The part that makes this a product rather than an experiment.

The AI provider used here gives away a limited amount for free: **500 requests per day**. One network test costs about 165 of them. So a whole day buys **three tests**.

The first version of this project was a script that planned forty tests and ran them all at once. It failed every single time it was run — not from a bug, but from arithmetic. Forty tests need thirteen days of allowance.

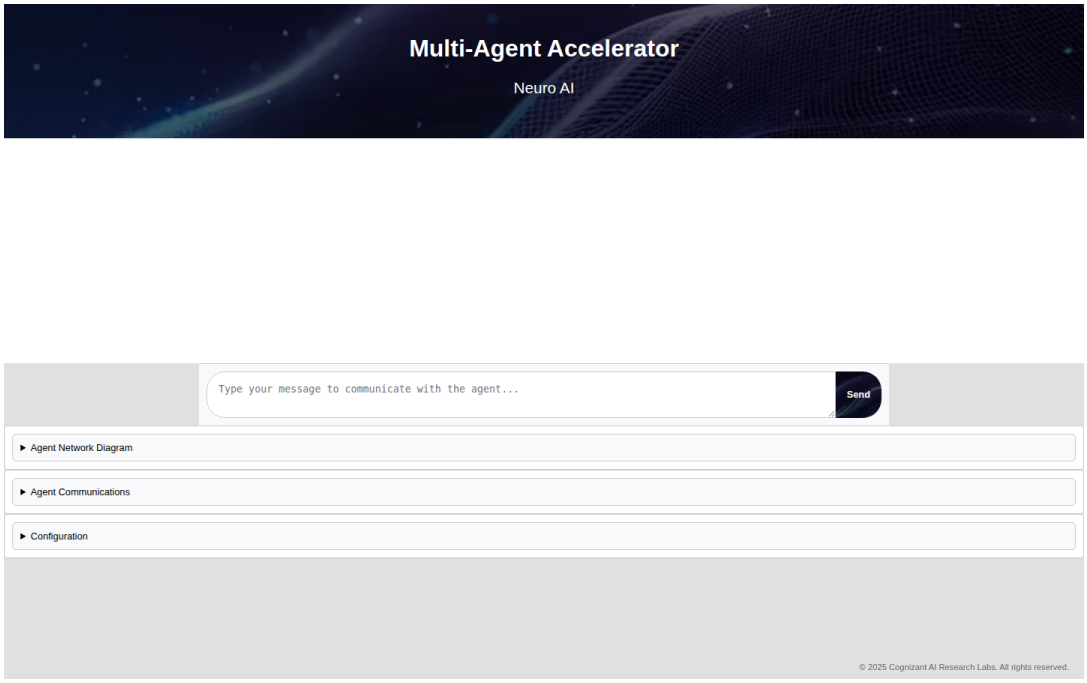
> **The change of mind that fixed it**
>
> The daily limit is not an obstacle to a service. It is its **rhythm**.
>
> Instead of one big run that dies, it now wakes up every hour, spends whatever allowance is left, writes down what it learned, and stops. Tomorrow it carries on. Over a few weeks it accumulates a collection of results that could never be bought in one afternoon — and nobody has to be watching.

What that requires, and why

It does this...	...because otherwise this happens
Saves after *every single test*, not at the end	It expects to be interrupted. Losing an eight-minute test that has already been paid for is the thing to avoid.
Writes to a temporary file, then renames it	If it is killed halfway through writing, it would come back to a half-written file and have lost weeks of results.
Takes a "lease" so only one copy runs at a time	Two copies would spend the same daily allowance twice and overwrite each other's results.
The lease expires by itself	If a copy is killed without releasing it, a lease that never expires would jam the service permanently.
Remembers exhausted limits *per day*, not forever	Marking a limit as permanently spent would leave the service dead after its first bad afternoon.
Stays silent unless it finds something better	Most wakes find nothing. A service that reports every wake trains you to ignore it — and then you ignore the one that matters.
Has no ability to delete or send anything	It is an unattended program with an AI in it, running every hour with nobody watching. It should not be able to do anything permanent.

You can talk to the winner

A search that ends with a winning identifier in a table has stopped one step short: the whole reason to measure designs is that one of them is better to *use*. One command writes the best-measured design into the framework as an ordinary agent, and it then answers questions in a web page like anything else — except this one was chosen by measurement instead of by somebody guessing.



The neuro-san web client in a browser, connected to the best-measured design.

> **It did not answer, and that is not hidden**
>
> The free daily allowance was spent when this was taken, so the agents replied with a quota error rather than a name. What it shows is still real: the winner is served, a browser reaches it, and the question travels through the actual agents. The answer needs allowance that day did not have. Saying so costs a sentence; implying otherwise would cost the credibility of every other number here.

Proof that it really does run by itself

This was not taken on trust. A real server was started and watched:

```
13:21:42  user_id=None     Found 1 periodic agent interactions
13:22:01  user_id=system   Received a optimizer request
13:23:00  user_id=system   Received a optimizer request
13:24:00  user_id=system   Received a optimizer request
```

user_id=system means there was no person and no app on the other end. The framework started those by itself, on schedule. That check now runs as a command anybody can repeat, and it *fails* if the wake-ups do not arrive.

What it has not done

Stated plainly, because this is the part people skip.

The search has not yet beaten the starting design

Only three real tests exist. Beating a baseline needs a collection of results, a collection needs allowance, and allowance arrives at three tests a day. That is the honest state of it. If it never beats the baseline, that will be reported as the answer — the reporting code renders a failure as readily as a success, and it distinguishes "we searched and found nothing better" from "we never searched", because those are different claims.

The predictor is currently no better than guessing

Trained on three examples, its measured accuracy at ranking is exactly chance, and the report prints that rather than hiding it. That is what three examples are worth. The machinery is right and the predictions are free; their *quality* depends on how many real results have accumulated — which is precisely why it runs every hour instead of once.

Other limits

- **One kind of task.** A design that wins at looking things up in documents need not win at anything else.
- **Searching AI designs is not a new idea.** Several research projects do versions of it. What none of them do is target neuro-san — and what neuro-san does not have is any way to score a design at all.
- **The comparison design is a faithful copy, not the literal output.** neuro-san's designer builds networks against its own tools and cannot see this pretend company, so its *shape* was reproduced rather than its exact text.

Seven things that were only learned by running it

Every one of these was a bug that made a *good* design look bad — the worst kind, because a search being fed wrong information still produces a smooth, convincing-looking graph.

The wrong agent was answering

The framework treats whichever agent is listed first as the one that receives the question. Listing them alphabetically meant one design was fronted by an arithmetic agent, which replied "you did not provide any numbers" to all 17 questions. It scored zero. It had never actually been run as designed — and neither had any other multi-agent measurement taken up to that point.

The cost measurement was flat

Scaled wrongly, so a network costing 300,000 and one costing 900,000 scored identically. The project would have claimed to balance cost against accuracy while ignoring cost entirely.

One waiting agent froze all the others

A pause written the wrong way stopped every agent in the program at once, and they all timed out together. Caught by an automated code check, not by a test — which is luck, so a test was added.

The first question set was too easy

A single agent scored 100%. There was nothing to improve. The document collection was tripled and harder questions added: an experiment whose starting point is already perfect measures nothing.

The free allowances are wildly uneven

Read out of the provider's own error messages rather than its documentation: some models allow 500 requests a day, some allow 20, and some allow none. A change that moved an agent onto a 20-a-day model was a guaranteed failure that would have been blamed on the design.

The service forgot which limit it had hit

It reported "nothing exhausted" straight after being cut off, because it was checking the wrong list of names. Left alone, every hourly wake for the rest of the day would have wasted its first requests rediscovering the same dead limit.

The published package was missing a piece

A library needed to produce these very PDFs was never declared as a requirement. It worked on the development machine because it happened to be installed there. Anyone else installing the project cleanly would have got a version that could not build its own report.

Words used here

In case any of them were new.

Word	What it means here
Agent	A small program given a job description in plain English, powered by an AI model.
Agent network	Several agents wired together, where one receives the question and asks the others for help.
neuro-san	Cognizant AI Lab's open-source framework for building agent networks. This project is built on top of it.
Topology	The shape of a network: how many agents, and who talks to whom.
ESP	Evolutionary Surrogate-assisted Prescription. Cognizant AI Lab's method: learn a cheap predictor, search against it for free, pay only for the best few. The wind-tunnel picture.
Surrogate / Predictor	The cheap stand-in that guesses a score without running anything. The simulator.
Fitness	The score a design gets — here, a combination of how many answers were right, how much it cost, and how many agents it used.
Token	Roughly a word. AI providers bill by these, so "tokens" and "cost" mean the same thing in this document.
Quota / rate limit	The provider's cap on how much you may use for free. Here: 500 requests per day, per model.
Baseline	The design you are trying to beat — in this case, the one neuro-san's own designer would have handed you.

Where to go next

The full technical write-up is in `docs/neuro-san-esp-Dossier.pdf` — every file, the deployment, and the captured evidence.
The code is at github.com/Sivakumarraj/neuro-san-esp.
To try it yourself with no account and no key, the search half runs for free: `make install` then `make offline`.

> **One sentence, if you only remember one**
> neuro-san can design an AI agent network but cannot tell you whether it is any good; this measures that, searches for a better one, and keeps searching by itself — and the first thing it measured was three designs with a 42% difference in cost, and a tie on correctness that turned out to be three different failures wearing the same score.